

Google Earth Engine Time Series Habitat Maps

Dylan Cowley

GEE Time Series (GitHub): <https://github.com/memlUQ/HabitatMapping>

Contents

1	Implementation	2
1.1	Accessing the Scripts	2
1.2	Required Input Datasets	2
1.2.1	Field Data	2
1.2.2	Imagery	2
1.2.3	Physical Attributes	2
1.3	Asset Structure	2
2	Habitat Mapping	3
2.1	Field Data Pre-Processing	3
2.2	Image Pre-Processing	3
2.3	Point Processing	6
2.4	Image Classification	9

1 Implementation

For the full **Google Earth Engine (GEE) Setup** go to:

<https://developers.google.com/earth-engine/guides/getstarted>

For detailed information regarding associated **Habitat Mapping Methodologies** see [Carrasco Rivera et al. \(2025\)](#) and [Cowley et al. \(2025\)](#).

1.1 Accessing the Scripts

All three image processing and classification scripts can be accessed via the [Marine Ecosystem Monitoring Lab](#) GitHub  page, here:

<https://github.com/memlUQ/HabitatMapping>

1.2 Required Input Datasets

1.2.1 Field Data

The processing scripts presented here utilise classified benthic photoquadrats collected in situ. Photoquadrats are classified using [ReefCloud](#) and output from here is condensed into the smaller subset of selected classes required for habitat mapping (see Section 2.1).

1.2.2 Imagery

Imagery used in [Carrasco Rivera et al. \(2025\)](#) and [Cowley et al. \(2025\)](#) is sourced from CASI, IKONOS, Quickbird 2, WorldView-2, WorldView-3, and [Planet](#). Any multi- or hyperspectral satellite or airborne imagery will work with this processing workflow, given the imagery contains spectral information for wavelengths between 400 and 700 nm or specific red, green, and blue bands.

1.2.3 Physical Attributes

The image pre-processing and classification scripts require some physical attributes. Depth is required for ease of use in the current format of the processing workflow, while wave and slope information is optional¹. Depth may also be omitted, but further modification of the processing workflow is required, in order to remove depth masking sections.

1.3 Asset Structure

An example of the GEE folder structure that works best with these processing scripts is presented below. The three **OUTPUT** folders and two **INPUT** folders should have the same names as below, though the name and structure of the parent directories is less critical, and can be modified from the *user input* section of each processing script (by modifying the `assetFolder` variable).

```
projects
├── user
│   ├── assets
│   │   ├── site_name
│   │   │   ├── CalVal
│   │   │   │   └── OUTPUT location for calibration and validation points from field data
│   │   │   ├── Classif
│   │   │   │   └── OUTPUT location for classified maps
│   │   │   ├── Processed_Imagery
│   │   │   │   └── OUTPUT location for processed imagery
│   │   │   ├── Raw_Imagery
│   │   │   │   └── INPUT location for raw imagery
│   │   │   ├── Raw_Points
│   │   │   │   └── INPUT location for field data
│   │   │   ├── dw
│   │   │   └── roi_geometry
```

¹Depth and wave height information should be included in **centimetres**.

2 Habitat Mapping

2.1 Field Data Pre-Processing

Classified photoquadrat data from [ReefCloud](#) is reorganised into the desired benthic classes required for creating habitat maps. Two scripts are used for the time series mapping that utilises the three Google Earth Engine scripts outlined in Section 2. The two scripts are GEE_Coral.R and GEE_Seagrass.R which are used for habitat mapping in coral reef and seagrass environments, respectively.

2.2 Image Pre-Processing

The raw satellite image is pre-processed in a GEE script (1.Image_Processing). In this process, statistical bands are calculated including: mean, median, standard deviation, texture measurements from the gray level co-occurrence matrix (GLCM), principal component analysis (PCA), and simple non-iterative clustering (SNIC) segmentation. Additionally, physical attribute layers are included from previous research: depth, slope, and waves. These physical layers can be omitted if not required.

The image processing script is divided into sections A through to G, detailed below. Variables that require modification or user input are indicated as `[variable]`.

Input

Data Required:

- [Raster asset in **TIFF** format] Multi- or hyperspectral satellite or airborne image raster
- [Raster asset in **TIFF** format] Physical layer raster, which contains depth, slope, and waves (or just depth) – depth and wave height should be in centimetres
- [Vector asset in **SHAPEFILE** format] Mapping area region of interest

User Specifications:

- Date information (as YYYYMM)
- Specification for red, green, and blue bands in the image
- Depth, slope, and waves band names
- Site name
- ROI file name
- Asset folder structure
- Image pixel size (in metres)

Output

Console Items (in the console):

- **Raw Image** – band information for the raw input imagery
- **Computed Statistics** – basic statistics from the raw input imagery
- **Depth Image** – band information for the physical layer

Display Items (on the map):

- **siteName ROI** – mapping area region of interest (ROI)
- **siteName: yearMonth** – true colour display for the input imagery
- **Image with Statistics** – basic statistics from the raw imagery
- **PC Image** – image with principal components
- **Physical Layer** – depth, slope, and waves raster
- **SNIC** – processed segmented image (this is the FINAL OUTPUT)

Output Items (requires confirmation in the **Tasks** window):

- [Raster asset in **TIFF** format] Processed segmented image is exported as an asset



A. User Input

- (0) Define the date for the image and field data in `[yearMonth]` as YYYYMM format². If multiple image dates are being used for the same location (with the same ROI), this is the only line that needs adjustment in this script
- (1) Define the names for the red `[R]`, green `[G]`, and blue `[B]` bands from the input image
- (2) Define the physical layers:
 - `[physicalLayer]` **Depth/Waves/Slope** raster layer name.
 - `[dwBands]` **Depth/Waves/Slope** band names within the raster.
 - `[dwBandNames]` New names for the **Depth/Waves/Slope** bands.
- (3) Enter the site name `[siteName]` which is used for the interim processing results in the display window
- (4) This section requires the following:
 - File name for the shapefile that contains geometry to constrain the region of interest (ROI) `[roiGeometry]`
 - The folder in which the input assets are contained and where the output assets will be saved `[assetFolder]`
 - Everything within the assets assignment `[assets]` should remain unchanged **except** for the assignment of pixel size relevant to the imagery used `[sensor_scale]`

B. Basic Display

- (1) Mapping extent defined using the specified ROI layer
- (2) Create a masking layer based on the ROI and display as `siteName ROI`
- (3) Display true colour image as `siteName Image: yearMonth` and print the band information in the console

C. Image Statistics

- (1) Clip image raster to ROI extent
- (2) Compute basic statistics (mean, median, standard deviation, minimum, and maximum) for each band
- (3) Print the statistics in the console
- (4) Add the statistical bands to the raster image
- (5) Display the image with the added statistics as `Image with Statistics`

D. Principal Components Analysis

- (1) Define bands to use for principal components analysis (PCA)
- (2) Calculate image covariance
- (3) Conduct PCA by finding eigenvalues and eigenvectors, then performing matrix multiplication (*optional*: display a single principal component)
- (4) Visualise the PCA output as `PC Image`, showing the first three PCs

E. Depth Masking

If no physical layer is used, section E.1 and E.2 should be removed, along with the `depth_g1cm` line in E.3 and `dept_var` line in E.4. Additionally, in section F.2, the following variables should be removed from the `run_snic` command: `DWimage`, `depth_g1cm`, `depth_var`, and `depth_mask`.

- (1) Load physical layer raster
- (2) Create depth mask to restrict ROI to the depth limit defined in the input raster, then visualise as `Physical Layer` with additional detail in the console

²The input satellite imagery and input field data should **both** be named as YYYYMM, saved in the correct asset paths



- (3) Compute the gray level co-occurrence matrix (GLCM) to define the textural bands across the depth, blue, and green layers
- (4) Compute additional statistics, standard deviation for depth and red bands
- (5) *Optional*: display the pre-segmentation image to check output

F. Segmentation

- (1) Define simple non-iterative clustering (SNIC) segmentation function
- (2) Apply the segmentation function and visualise as "SNIC" item
- (3) *Optional*: remove clusters to aid display and checking

G. Export Image

- (1) Export the processed image as an asset to the Processed_Imagery folder
- (2) *Optional*: export the processed image to Google Drive if required

2.3 Point Processing

The image output from 1. Image_Processing is combined with the field data in this script (2.Points_Processing). The field data is separated into calibration/training and validation datasets here and saved as output to the CalVal folder as a GEE asset. Calibration and validation data is assigned to each point from the processed image created in the first script.

The point processing script requires the output from the image processing script and is divided into sections A through to F, detailed below. Variables that require modification or user input are indicated as [variable].

Input

Data Required:

- [Raster asset in **TIFF** format] Multi- or hyperspectral satellite or airborne image raster
- [Raster asset in **TIFF** format] Processed segmented image from the script 1. Image_Processing
- [Vector asset in **SHAPEFILE** format] Mapping area region of interest
- [Point asset in **CSV** format] Field data as points, requiring at least latitude, longitude, and mapping class

User Specifications:^a

- Date information (as YYYYMM)
- Specification for red, green, and blue bands in the image
- Number of mapping classes
- Site name
- ROI file name
- Asset folder structure
- Image pixel size (in metres)

^aNote that there is specification for cover type, which varies between *species* or *percov*. This can be removed if only one map type is being produced by removing all references to `coverType` throughout and changing the `classChoice` variable to be only equal to `Class_num`.

Output

Console Items (in the console):

- **Raw Image** – band information for the raw input imagery
- **PI Bands** – band information for the processed segmented image
- **Training dataset information** – this includes:
 - Total number of training points
 - Classes in the training point dataset
 - Classes in the *calibration* dataset
 - Class counts in the *calibration* dataset
 - Classes in the *validation* dataset
 - Class counts in the *validation* dataset
 - Total number of calibration features
 - Total number of validation features

Display Items (on the map):

- **siteName ROI** – mapping area region of interest (ROI)
- **siteName: yearMonth** – true colour display for the input imagery
- **Processed Segmented Image: yearMonth** – processed segmented image from 1. Image_Processing
- **Training Points: yearMonth** – field data locations
- **Calibration points** – the locations of the proportion of field data assigned to calibration (this is the FINAL OUTPUT)
- **Validation points** – the locations of the proportion of field data assigned to validation (this is the FINAL OUTPUT)
- **Training Data** – summary of all training points

Output Items (requires confirmation in the **Tasks** window):

- [Point asset in **TABLE** format] Calibration points
- [Point asset in **TABLE** format] Validation points



A. User Input

- (0) Define the date for the image and field data in `[yearMonth]` as YYYYMM format. If multiple image dates are being used for the same location (with the same ROI and benthic classes), this is the only line that needs adjustment in this script
- (1) Define the names for the red `[R]`, green `[G]`, and blue `[B]` bands from the input image
- (2) Define benthic cover type (mapping classes), as in the field data (which shares the same name as the imagery as YYYYMM) uploaded as an asset in the `Raw_Points` folder. The mapping classes will be read from the column `Class_num` within the associated field data spreadsheet
- (3) Enter the site name `[siteName]` which is used for the interim processing results in the display window
- (4) This section requires the following:
 - File name for the shapefile that contains geometry to constrain the region of interest (ROI) `[roiGeometry]`
 - The folder in which the input assets are contained and where the output assets will be saved `[assetFolder]`
 - Everything within the assets assignment `[assets]` should remain unchanged **except** for the assignment of pixel size relevant to the imagery used `[sensor_scale]`

B. Load Assets

- (1) Mapping extent defined using the specified ROI layer
- (2) Create a masking layer based on the ROI and display as `siteName ROI`
- (3) Display true colour image as `siteName Image: yearMonth` and print the band information in the console
- (4) Load the processed image asset created in 1. `Image_Processing` and display as `Processed Segmented Image` to check, along with band information in the console
- (5) Load the training data asset from the `Raw_Points` folder and print the number of training points + benthic classes in the training dataset into the console
- (6) Read mapping class information from the field data (in the `Class_num` column)

C. Define Calibration Points

- (1) Define the proportion of field data to be used for calibration of the model (this is defined as 80% here)
- (2) Set a seed (to allow for reproducibility) to randomly assign points
- (3) Print the number of calibration points for each mapping class in the console
- (4) Display the calibration points in the display window

D. Define Validation Points

- (1) Define the proportion of field data to be used for validation of the model (this is defined as 20% here)
- (2) Set a seed (to allow for reproducibility) to randomly assign points
- (3) Print the number of validation points for each mapping class in the console
- (4) Display the validation points in the display window

E. Assign Training Data

- (1) Sample the calibration data from the processed image using the defined calibration points
- (2) Sample the validation data from the processed image using the defined validation points
- (3) Display all the training data in the display window and print the number of calibration/validation features in the console

F. Export Points

- (1) Define export names and locations



- (2) Export the calibration and validation datasets as assets to the `CalVal` folder
- (3) *Optional*: export the calibration and validation datasets to Google Drive if required

2.4 Image Classification

The image output from 1. Image_Processing is combined with the training data from (2.Points_Processing) to produce a classified image in 3. Classification. The selected satellite image is classified into the defined benthic classes using the calibration and validation data and a random forest classifier.

The point processing script requires the output from both the image processing and points processing scripts and is divided into sections A through to F, detailed below. Variables that require modification or user input are indicated as [variable].

Input

Data Required:

- [Raster asset in **TIFF** format] Multi- or hyperspectral satellite or airborne image raster
- [Raster asset in **TIFF** format] Physical layer raster, which contains depth, slope, and waves (or just depth) – depth and wave height should be in centimetres
- [Vector asset in **SHAPEFILE** format] Mapping area region of interest
- [Point asset in **TABLE** format] Calibration points
- [Point asset in **TABLE** format] Validation points

User Specifications:^a

- Date information (as YYYYMM)
- Specification for red, green, and blue bands in the image
- Number of mapping classes
- Colour palette for displaying the classified image (in the script colour_pals)^b
- Site name
- ROI file name
- Asset folder structure
- Image pixel size (in metres)

^aNote that there is specification for cover type, which varies between *species* or *percov*. This can be removed if only one map type is being produced by removing all references to `coverType` throughout and changing the `classChoice` variable to be only equal to `Class_num`.

^bHere the `colpals`, `palMax`, `palChoice`, and `palList` variables will also have to be adjusted to suit cases with different mapping classes.

Output

Console Items (in the console):

- **Confusion Matrix** – confusion matrix for all mapping class combinations
- **Overall Accuracy** – overall accuracy for the classification
- **Producers Accuracy** – producer's accuracy for each mapping class
- **Consumers Accuracy** – consumer's accuracy for each mapping class

Display Items (on the map):

- **siteName: yearMonth** – true colour display for the input imagery
- **Classification: yearMonth** – classified image (this is the FINAL OUTPUT)

Output Items (requires confirmation in the **Tasks** window):

- [Raster asset in **TIFF** format] Classified image is exported as an asset
- [Raster asset in **TIFF** format] Classified image is exported to Google Drive
- [Point asset in **CSV** format] Classification accuracies are exported to Google Drive



A. User Input

- (0) Define the date for the image and field data in `[yearMonth]` as YYYYMM format. If multiple image dates are being used for the same location (with the same ROI and benthic classes), this is the only line that needs adjustment in this script
- (1) Define the names for the red `[R]`, green `[G]`, and blue `[B]` bands from the input image
- (2) Define benthic cover type (mapping classes), the number of benthic classes and the colour palette for mapping
- (3) Enter the site name `[siteName]` which is used for the interim processing results in the display window
- (4) This section requires the following:
 - File name for the shapefile that contains geometry to constrain the region of interest (ROI) `[roiGeometry]`
 - The folder in which the input assets are contained and where the output assets will be saved `[assetFolder]`
 - Everything within the assets assignment `[assets]` should remain unchanged **except** for the assignment of pixel size relevant to the imagery used `[sensor_scale]`

B. Basic Display

- (1) Mapping extent defined using the specified ROI layer
- (2) Create a masking layer based on the ROI and display as `siteName ROI`
- (3) Display true colour image as `siteName Image: yearMonth` and print the band information in the console
- (4) Load the processed image produced in 1. Image_Processing and clip the image to the specified maximum depth – if depth masking is not required, the `[depth_max]` and `[PI_clip]` lines can be removed and any reference to `PI_clip` further in the script could be replaced with `PI`

C. Image Classification

- (1) Load calibration/validation data produced from 2. Points_Processing
- (2) Train a Random Forest classifier using the calibration/validation data and the processed segmented image
- (3) Run the trained classifier on the processed segmented image and display the classification results as `Classification: yearMonth`
- (4) Create and display a legend using the predefined colour palette

D. Export Classified Image

- (1) Export the classified image as an asset
- (2) Export the classified image to Google Drive

E. Accuracy Assessment

- (1) Import reference data from 2. Points_Processing for accuracy assessment
- (2) Create a confusion matrix based on the classified image and the reference data
- (3) Print the accuracy metrics in the console, including the confusion matrix, overall accuracy, and producer's/consumer's accuracies

F. Export Accuracies

- (1) Create a feature to store accuracy information
- (2) Export basic accuracy information to Google Drive in a .csv format



References

- Carrasco Rivera, D., Diederiks, F., Hammerman, N., Staples, T., Kovacs, E., Markey, K., and Roelfsema, C. (2025). Remote sensing reveals multidecadal trends in coral cover at heron reef, australia. *Remote Sensing*, 17(7):1286.
- Cowley, D., Carrasco Rivera, D., Smart, J., Hammerman, N., Golding, K., Diederiks, F., Hafizt, M., and Roelfsema, C. (2025). Insights into seagrass persistence and resilience from decades of satellite monitoring. *Remote Sensing*, X(X):X.